

Documentation CI/CD | Projet 7

Documentation CI/CD du projet "Mettez en œuvre l'intégration et le déploiement continu d'une application full-stack JavaScript" - Option A

Projet réalisé par Dufrène Valérian, au cours du cursus Lead Dev JS, entre Avril 2026 et Octobre 2026

Diagnostic synthétique

Contexte du projet

Le projet **webdevoo-lead** est une application web full-stack développée avec :

- Nuxt 4
- Vue 3
- TypeScript
- Bun
- Drizzle ORM
- MySQL 8.0
- Vitest
- Playwright
- GitHub Actions
- Semantic Release

Ce projet répond à un besoin de l'entreprise **Webdevoo**, gérée par **Dufrène Valérien**, immatriculée **84959557400013**, en lien avec l'apport d'affaires.

Besoin initial exprimé

J'ai besoin d'une plateforme permettant à des apporteurs d'affaires de reporter simplement les prospects actuels, de manière centralisée, en ayant la possibilité de négocier leur taux de commission pour l'apport.

Chaque prospect déposé doit passer par 4 étapes : **en attente, validé, clôturé/payé, refusé.**

Mise en place CI/CD

L'objectif de la démarche CI/CD est de garantir que chaque modification du code puisse être automatiquement :

- récupérée
- vérifiée statiquement
- testée unitairement
- testée fonctionnellement
- compilée
- versionnée
- produite sous forme d'artefact reproductible

Le projet dispose d'une **CI/CD fonctionnelle** dans **GitHub Actions**, mais ne possède pas de déploiement de production automatisé.

Ce choix est volontaire : le projet est réalisé dans le cadre d'un cursus Bac+5 et l'objectif est de démontrer la mise en œuvre d'une chaîne CI/CD complète sans nécessiter d'infrastructure de production.

L'architecture cible d'une production pourrait toutefois être adaptée à un hébergement classique de type PlanetHoster avec :

Nuxt



Serveur d'hébergement



MySQL

La conteneurisation Docker est utilisée dans la CI pour isoler notamment l'environnement MySQL nécessaire aux tests E2E, mais elle n'est pas considérée comme obligatoire pour le futur hébergement de production.

Problématiques identifiées

Les principaux objectifs sont :

- détecter rapidement les régressions
- garantir la reproductibilité des environnements de test
- automatiser les contrôles qualité
- limiter les erreurs humaines
- assurer la traçabilité des versions
- permettre la reproductibilité du projet et de ses fonctionnalités
- surveiller les dépendances
- mesurer progressivement la performance du processus de développement avec les indicateurs DORA

Mise en place technique CI/CD

Architecture

Le pipeline GitHub Actions fonctionne comme une chaîne de validation :

Commit / Pull Request

↓

Installation Bun

↓

Installation dépendances

↓

Tests unitaires Vitest

↓

Tests E2E Playwright

↓

Build Nuxt

↓

Semantic Release

Le pipeline est déclenché sur les branches :

- dev
- staging
- master

ainsi que sur les Pull Requests ciblant ces branches.

Analyse statique

Le pipeline commence par :

```
npx tsc --noEmit
```

Cette étape permet de vérifier que le projet respecte les contraintes TypeScript sans générer de fichiers JavaScript.

Elle permet notamment de détecter :

- erreurs de typage
- propriétés inexistantes
- incompatibilités entre types
- erreurs dans les interfaces utilisées par l'application

Preuve technique :

- *name: Type check & Lint*

```
run: npx tsc --noEmit
```

Tests unitaires

Les tests unitaires sont exécutés avec Vitest, via la commande :

```
bun run test:unit
```

Ils permettent de contrôler les fonctions et composants isolément avant d'exécuter les tests fonctionnels.

Preuve technique :

- *name: Run Unit Tests*

```
run: bun run test:unit
```

Tests E2E

Les tests end-to-end sont exécutés avec Playwright.

L'environnement de test utilise un service MySQL Dockerisé :

services:

mysql:

```
image: mysql:8.0
```

La base de données est donc créée dans un environnement isolé et temporaire.

Le pipeline exécute ensuite la commande :

```
bun run db:push
```

puis la commande :

`npx playwright test`

Les tests permettent de vérifier les parcours fonctionnels complets de l'application, comme par exemple le segment :

Inscription

↓

Connexion

↓

Création d'un Lead

↓

Vérification

↓

Déconnexion

Conteneurisation et déploiement

Conteneurisation utilisée dans la CI

La conteneurisation est actuellement utilisée principalement pour garantir un environnement reproductible lors des tests E2E. Le job Playwright utilise la version jammy, pour fluidifier le lancement en récupérant une image moins lourde, car non accompagnée d'une pléthore de dépendances par défaut :

container:

image: mcr.microsoft.com/playwright:v1.61.1-jammy

et MySQL :

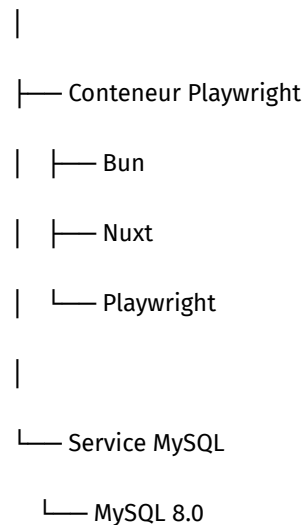
services:

mysql:

image: mysql:8.0

L'environnement de test garantit que les tests ne dépendent pas directement de l'installation MySQL de la machine :

GitHub Actions



Déploiement actuel

Le projet ne réalise pas de déploiement automatique en production.

Le pipeline produit cependant un artefact Nuxt :

- *name: Upload production artifact*

uses: actions/upload-artifact@v4

with:

name: nuxt-production-build

path: .output/

Le dossier **.output/** constitue donc le résultat compilé de l'application.

Architecture de production cible

Dans le cadre d'un futur déploiement, l'application pourrait être hébergée sur un serveur classique de type PlanetHoster.

Le processus serait :

GitHub

↓

Tests

↓

Build

↓

Artifact .output/

↓

Serveur PlanetHoster

↓

Nuxt

↓

MySQL

La conteneurisation Docker ne constitue donc pas une obligation pour la production.

Rollback

Chaque version produite est traçable grâce à Git et Semantic Release.

Par exemple :

v1.3.0

v1.4.0

v1.5.0

En cas de problème sur une version, le principe de rollback consiste à restaurer une version antérieure.

v1.5.0

↓

Incident

↓

Rollback

↓

v1.4.0

Dans le contexte actuel, ce mécanisme est **documenté mais non automatisé**, puisqu'aucun environnement de production n'est déployé.

L'historique Git et les artefacts GitHub constituent les éléments de traçabilité permettant de reconstruire une version antérieure.

Plan de testing périodique

Tests continus

Les tests sont actuellement exécutés lors des modifications du projet :

Push / Pull Request

↓

TypeScript

↓

Vitest

↓

Playwright

↓

Build

Cela constitue le **Continuous Testing**.

Tests planifiés

Une évolution possible consiste à créer un workflow indépendant :

.github/

└─ *workflows/*

├─ *ci-cd.yml*

└─ *periodic-tests.yml*

L'objectif est de ne pas modifier le pipeline fonctionnel existant, car le projet actuel étant destiné à être géré par une seule personne est largement sécurisé par la pipeline au commit.

Le workflow périodique pourrait être déclenché avec :

on:

schedule:

- *cron: "0 2 * * *"*

Il exécuterait les tests E2E dans un environnement propre :

Chaque nuit



GitHub Actions



MySQL Docker



db:push



Playwright



Résultat

Cette approche permettrait de détecter une régression qui ne serait pas nécessairement liée à un nouveau commit.

Différence entre CI et tests périodiques

CI

Modification du code



Tests

Objectif : Vérifier qu'une modification ne casse pas le projet.

Tests périodiques

Tous les jours



Tests

Objectif : Vérifier que l'environnement et l'application restent fonctionnels dans le temps.

Plan de sécurité

Gestion des secrets

Les secrets ne sont pas stockés dans le dépôt Git.

Les informations sensibles sont stockées dans les **GitHub Secrets**.

Par exemple :

```
DB_ROOT_PASSWORD: ${{ secrets.DB_ROOT_PASSWORD }}
```

Cela permet d'éviter l'exposition des identifiants de base de données dans le code source.

/!\ : Au niveau du projet, il est possible également, si des secrets hébergés ne sont pas utilisables, de créer un fichier **.env** à la racine de l'application, pour fournir ces secrets.

Variables d'environnement

L'application utilise des variables d'environnement pour séparer la configuration du code, dont les valeurs peuvent varier selon l'environnement en place.

URL de connexion utilisée par Drizzle ORM (format standard mysql2)

DATABASE_URL="mysql://pseudo:password@localhost:3306/votre_bdd"

DB_HOST=127.0.0.1

DB_USER=pseudo

DB_PASSWORD=password

DB_NAME=your_db

DB_PORT=3306

Clé secrète robuste pour signer les Access Tokens (JWT)

JWT_ACCESS_SECRET="VotreSignatureJWT@12"

Clé secrète pour signer ou chiffrer les Refresh Tokens

JWT_REFRESH_SECRET="VotreSignatureREFRESH@12"

Environnement d'exécution (development, production)

NODE_ENV="development"

Port d'écoute du serveur Nuxt (par défaut 3000)

PORT=3000

Protection contre les injections SQL

L'application utilise Drizzle ORM avec MySQL.

L'utilisation d'un ORM typé permet de structurer les accès à la base de données et de limiter les risques liés à la construction manuelle de requêtes SQL, en utilisant les requêtes préparées.

L'utilisation de Drizzle ORM couplé au pilote **mysql2** garantit une sécurité "by design" en éliminant les risques d'injections SQL.

Contrairement aux requêtes manuelles sujettes aux failles de concaténation, l'ORM s'appuie nativement sur des requêtes préparées et un typage strict, traitant chaque saisie utilisateur comme un paramètre isolé que le moteur de base de données ne peut pas interpréter comme du code exécutable.

De plus, ce typage de bout en bout associé à drizzle-kit assure une maîtrise totale des évolutions de la base de données.

Les schémas sont validés dès la compilation par TypeScript et synchronisés automatiquement dans le pipeline CI/CD, garantissant une parité parfaite entre l'environnement local et la production sans risque d'écart de structure.

Authentification

Les mots de passe utilisateurs sont protégés avant stockage grâce à **bcryptjs**.

Le mot de passe en clair n'est donc pas stocké directement en base.

Hachage robuste des mots de passe :

Les mots de passe des utilisateurs ne sont jamais stockés en clair en base de données.

Avant toute insertion (comme vérifié dans les tests E2E et l'API d'inscription), ils sont transformés via la bibliothèque **bcryptjs**, qui applique un algorithme de salted hashing.

Cela protège efficacement les utilisateurs contre les attaques par rainbow tables ou par force brute en cas de compromission de la base.

Gestion des sessions :

L'utilisation de **JWT** permet de sécuriser l'authentification et d'identifier de manière unique et chiffrée les utilisateurs connectés lors de leurs requêtes aux API, sans exposer leurs identifiants à chaque échange.

Cette application utilise un **access_token**, qui expire régulièrement, ainsi qu'un **refresh_token**, qui permet de sécuriser l'autorisation de renouvellement des **access_token**.

Dépendances

Le projet utilise un lockfile Bun afin de garantir une installation reproductible des dépendances.

Le **Dependency Graph GitHub** est activé.

Dependabot est également activé avec une fréquence mensuelle.

Le processus est donc :

Dependabot

↓

Analyse des dépendances

↓

Mise à jour disponible

↓

Pull Request

↓

CI

↓

Tests

Il n'est donc pas nécessaire d'ajouter un mécanisme supplémentaire de mise à jour automatique.

KPI et métriques

Le projet s'appuie sur les **GitHub Insights** pour analyser les métriques liées au développement, basé sur les indicateurs **DORA**, comme demandé dans le cahier des charges du projet.

Deployment Frequency

Mesure la fréquence des mises à disposition/déploiements.

Exemple :

18 releases sur un mois

Dans le contexte actuel, il faut préciser que le projet ne possède pas de déploiement de production automatisé.

Les releases Git et Semantic Release constituent donc principalement les éléments de traçabilité disponibles.

Lead Time for Changes

Mesure le délai entre une modification et sa mise à disposition.

Commit

↓

CI

↓

Build / Release

Par exemple :

Commit : 10:02

Release : 10:17

Lead Time : 15 minutes

GitHub permet d'exploiter les informations relatives aux commits, Pull Requests et workflows pour analyser ce délai.

Change Failure Rate

Le Change Failure Rate correspond à la proportion de changements ayant provoqué un incident.

Déploiements ayant provoqué un incident

Total des déploiements

Par exemple :

20 déploiements

2 incidents

CFR = 10 %

Dans le projet actuel, aucun système de production ou de gestion automatique d'incidents n'est en place, car je n'avais pas de budget financier à allouer à cette tâche et les outils les plus adaptés sont sous forme de SaaS à abonnement mensuel et annuel.

Cette métrique doit être interprétée, de manière humaine, grâce aux checkpoints du workflow Github Actions en place.

Les incidents sont représentés par les tests non validés par les derniers changements dans le code, et ne passent donc pas la **CI**.

Time to Restore

Le Time to Restore mesure le temps nécessaire pour restaurer le service après un incident.

Détection

↓

Correction / rollback

↓

Service restauré

Par exemple :

Incident : 14:03

Restauration : 14:21

TTR = 18 minutes

Cette métrique nécessite un environnement de production ou de préproduction supervisé pour être mesurée automatiquement, il n'est donc pas exposable sur ce projet de formation.

Concernant la procédure qui serait à appliquer en production :

Détection des incidents sur le serveur de production (les incidents précédents seraient évités par la CI)

↓

Rollback automatisé sur la dernière release stable

↓

Lancement d'une alerte automatisé (mail / sms) pour prévenir l'équipe du rollback effectué

↓

Service restauré (avec dernières modifications en attente de correction et de redéploiement)

Analyse des métriques

Les métriques ne doivent pas uniquement être collectées : elles doivent permettre d'identifier les axes d'amélioration.

Exemple d'analyse :

KPI

***Deployment
Frequency***

Lead Time

Change Failure Rate

Time to Restore

Analyse

Permet d'évaluer la capacité à livrer régulièrement

Permet d'identifier les étapes ralentissant la livraison

Permet d'évaluer la fiabilité des releases

Permet d'évaluer l'efficacité de la résolution d'incidents

Dans le projet actuel, GitHub Insights constitue la source principale d'analyse.

Les métriques doivent être interprétées en tenant compte du contexte : **le projet est évalué dans un environnement CI/CD sans production réellement déployée.**

Les métriques de déploiement et de restauration sont donc présentées comme des indicateurs applicables à l'architecture cible.

Plan de sauvegarde des données

Base de données CI

La base MySQL utilisée par les tests E2E est éphémère.

GitHub Actions

↓

MySQL Docker

↓

Tests

↓

Fin du job

↓

Environnement supprimé

Ces données n'ont aucune valeur métier et ne nécessitent donc pas de sauvegarde.

Base locale Docker

Dans un environnement local, MySQL peut utiliser un volume Docker afin de conserver les données entre les redémarrages.

Cependant, un volume Docker n'est pas une sauvegarde.

Une sauvegarde indépendante reste nécessaire pour protéger les données.

Au niveau de la procédure, on pourrait mettre en place ces différentes étapes :

- **Exports logiques périodiques** : Plutôt que de dépendre de la simple persistance des fichiers de la base de données, la stratégie repose sur l'extraction régulière de dumps logiques (fichiers SQL compressés) de l'instance MySQL. Cela isole la sauvegarde de l'état physique du serveur.
- **Automatisation par tâche planifiée (Cron)** : L'exécution des sauvegardes est automatisée à intervalles réguliers (par exemple, de manière quotidienne en dehors des heures de forte activité) via un planificateur de tâches système. Cette automatisation supprime le risque d'oubli humain.
- **Politique de rétention** : Un mécanisme de nettoyage automatique est associé aux sauvegardes pour conserver uniquement un historique glissant (par exemple, les 7 derniers jours). Cela permet de protéger l'espace disque tout en conservant une profondeur de restauration suffisante en cas d'incident.
- **Traçabilité et procédure de restauration** : Chaque sauvegarde est archivée dans un répertoire dédié et sécurisé. Une procédure de restauration documentée et testée périodiquement garantit que les archives peuvent être réinjectées rapidement en cas de corruption ou de perte accidentelle de données.

Procédure de sauvegarde cible

Pour une future production, le processus recommandé serait :

MySQL

↓

mysqldump

↓

Fichier SQL

↓

Stockage externe

↓

Rotation

Pour l'environnement de production qui pourrait être envisagé par Webdevoo, qui se tournerait vers **The World**, de **Planethoster**, on y dispose d'un système par défaut de **backups** sur **90 jours glissants**, pour le code source et la base de données.

En appliquant cette stratégie en production, on prévoit :

- une sauvegarde quotidienne
- la conservation pendant 7 jours minimum
- un stockage distinct du serveur applicatif
- une vérification régulière de la restauration

Plan de mises à jour

Mise à jour des dépendances

La surveillance des dépendances est assurée par :

- Dependency Graph ;
- Dependabot ;
- lockfile Bun.

Dependabot est configuré avec une fréquence mensuelle.

Dependabot

↓

Détection d'une mise à jour

↓

Pull Request

↓

CI

|— *TypeScript*

|— *Vitest*

|— *Playwright*

|— *Build*

↓

Validation

Une mise à jour qui provoque un échec de la CI peut ainsi être identifiée avant intégration.

Mise à jour de l'application

Les modifications de l'application suivent le même processus :

Développement

↓

Commit

↓

Pull Request

↓

CI

↓

Tests

↓

Build

↓

Merge

↓

Semantic Release

Le versionnement permet de conserver une traçabilité des évolutions.

Synthèse de l'état du projet

Le projet dispose actuellement d'une chaîne CI/CD fonctionnelle reposant sur GitHub Actions.

Fonctionnalités réellement implémentées

- GitHub Actions
- analyse TypeScript
- tests unitaires Vitest
- tests E2E Playwright
- MySQL Dockerisé pour les tests
- build Nuxt
- Semantic Release
- gestion des versions
- génération d'artefacts
- GitHub Secrets
- Dependency Graph
- Dependabot mensuel
- Git / GitHub pour la traçabilité

Fonctionnalités documentées comme architecture cible

- déploiement production
- monitoring
- health check
- rollback automatisé
- sauvegarde de production
- restauration
- métriques DORA de production